JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.logging

Class SocketHandler

java.lang.Object
  java.util.logging.Handler
    java.util.logging.StreamHandler
      java.util.logging.SocketHandler

```
public class SocketHandler
extends StreamHandler
```

Simple network logging `Handler`.

`LogRecords` are published to a network stream connection. By default the `XMLFormatter` class is used for formatting.

Configuration: By default each `SocketHandler` is initialized using the following `LogManager` configuration properties where `<handler-name>` refers to the fully-qualified class name of the handler. If properties are not defined (or have invalid values) then the specified default values are used.

- <handler-name>.level specifies the default level for the `Handler` (defaults to `Level.ALL`).
- <handler-name>.filter specifies the name of a `Filter` class to use (defaults to no `Filter`).
- <handler-name>.formatter specifies the name of a `Formatter` class to use (defaults to `java.util.logging.XMLFormatter`).
- <handler-name>.encoding the name of the character set encoding to use (defaults to the default platform encoding).
- <handler-name>.host specifies the target host name to connect to (no default).
- <handler-name>.port specifies the target TCP port to use (no default).

For example, the properties for `SocketHandler` would be:

- java.util.logging.SocketHandler.level=INFO

- java.util.logging.SocketHandler.formatter=java.util.logging.SimpleFormatter

For a custom handler, e.g. com.foo.MyHandler, the properties would be:

- com.foo.MyHandler.level=INFO
- com.foo.MyHandler.formatter=java.util.logging.SimpleFormatter

The output IO stream is buffered, but is flushed after each `LogRecord` is written.

Since:

`1.4`

Constructor Summary

Constructors

Constructor	Description
`SocketHandler()`	Create a `SocketHandler`, using only `LogManager` properties (or their defaults).
`SocketHandler(String host, int port)`	Construct a `SocketHandler` using a specified host and port.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
`void`	`close()`	Close this output stream.
`void`	`publish(LogRecord record)`	Format and publish a `LogRecord`.

Methods inherited from class java.util.logging.StreamHandler

`flush`, `isLoggable`, `setEncoding`, `setOutputStream`

Methods inherited from class java.util.logging.Handler

`getEncoding`, `getErrorManager`, `getFilter`, `getFormatter`, `getLevel`, `reportError`, `setErrorManager`, `setFilter`, `setFormatter`, `setLevel`

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

SocketHandler

```
public SocketHandler()
             throws IOException
```

Create a SocketHandler, using only LogManager properties (or their defaults).

Throws:

IllegalArgumentException - if the host or port are invalid or are not specified as LogManager properties.

IOException - if we are unable to connect to the target host and port.

SocketHandler

```
public SocketHandler(String host,
                     int port)
             throws IOException
```

Construct a SocketHandler using a specified host and port. The SocketHandler is configured based on LogManager properties (or their default values) except that the given target host and port arguments are used. If the host argument is empty, but not null String then the localhost is used.

Parameters:

host - target host.

port - target port.

Throws:

IllegalArgumentException - if the host or port are invalid.

IOException - if we are unable to connect to the target host and port.

Method Detail

close

```
public void close()
           throws SecurityException
```

Close this output stream.

Overrides:

`close` in class `StreamHandler`

Throws:

`SecurityException` - if a security manager exists and if the caller does not have LoggingPermission("control").

publish

```
public void publish(LogRecord record)
```

Format and publish a LogRecord.

Overrides:

`publish` in class `StreamHandler`

Parameters:

`record` - description of the log event. A null record is silently ignored and is not published

Submit a bug or feature

For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.